1. objectifs du tp

1.1 architecture de l'application

Nous avons créer deux micro services en python avec Flask :

- le micro service "clients" qui gère les informations des clients

- le micro service 'commandes" qui gère les informations des commandes

le but est de faire communiquer les deux micro services : à partir d'une commande, qui contient l'id du client, nous voulons récupérer les informations de ce client. Le microservice Commandes interroge donc le microservice Clients avec une API REST pour récupérer les infos du client d'une commande (ainsi via des requêtes HTTP)

Les deux tournent dans des conteneurs Docker reliés par un réseau Docker. Un conteneur est un environnement isolé, qui contient un programme, et tout ce dont il a besoin pour fonctionner. Un réseau docker est un réseau prové virtuel entre conteneurs. Ainsi les conteneurs branchés sur le même réseau peuvent communiquer entre eux, en s'appelant par leur nom.

1.2 création du projet

(*pour vérifier lancer depuis votre dossier tp-microservices dans le terminal : ls -R)*

Notre architecture suit celle demandée dans la consigne. Chaque microservice a son propre dossier avec trois fichiers :

- app.py : le code du microservice (le serveur Flask et ses routes)
- requirements.txt : les bibliothèques python nécessaires (flask pour clients, flask et requests pour commandes)
- Dockerfile : la méthode pour construire l'image Docker du microservice

```
./projet/tp-microservices$ ls -R
.:
clients  commandes

./clients:
app.py  Dockerfile  requirements.txt

./commandes:
app.py  Dockerfile  requirements.txt
```

1.3 api rest du micro service clients

Le micro service clients est un serveur Flasks sur le port 8080. Il contient la liste des clients, et expose deux routes :

GET /clients : retourne tous les clients
GET /clients/<id> : retourne un client selon son id

le code de app.py :

```python
from flask import Flask, jsonify

app = Flask(__name__)

clients = [
    {"id":1, "nom": "Alice", "pays": "France"},
    {"id":2, "nom": "Alex", "pays": "Belgique"},
    {"id":3, "nom": "Madeleine", "pays": "France"}
]

#1.3 : GET /clients : retourne tous les clients
@app.route("/clients")
def deg_clients():
    return jsonify(clients)

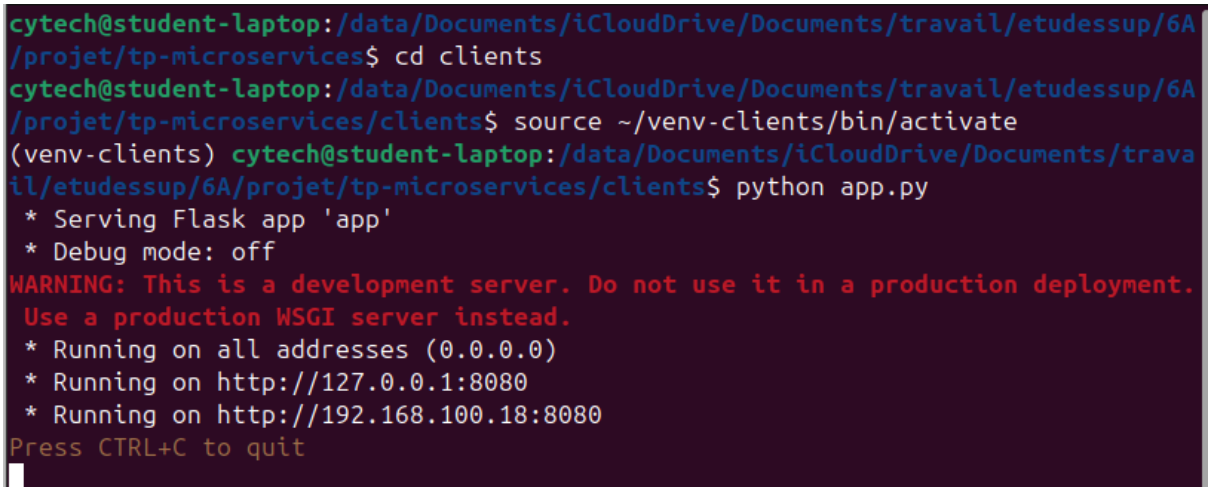
#1.3 : GET /clients/<id> : retourne un client
@app.route("/clients/<int:id>")
def deg_client(id):
    for client in clients:
        if client["id"]== id:
            return jsonify(client)

    # 2.2 gestion des erreurs : client inexistant
    return jsonify({"erreur": "Client introuvable"}), 404

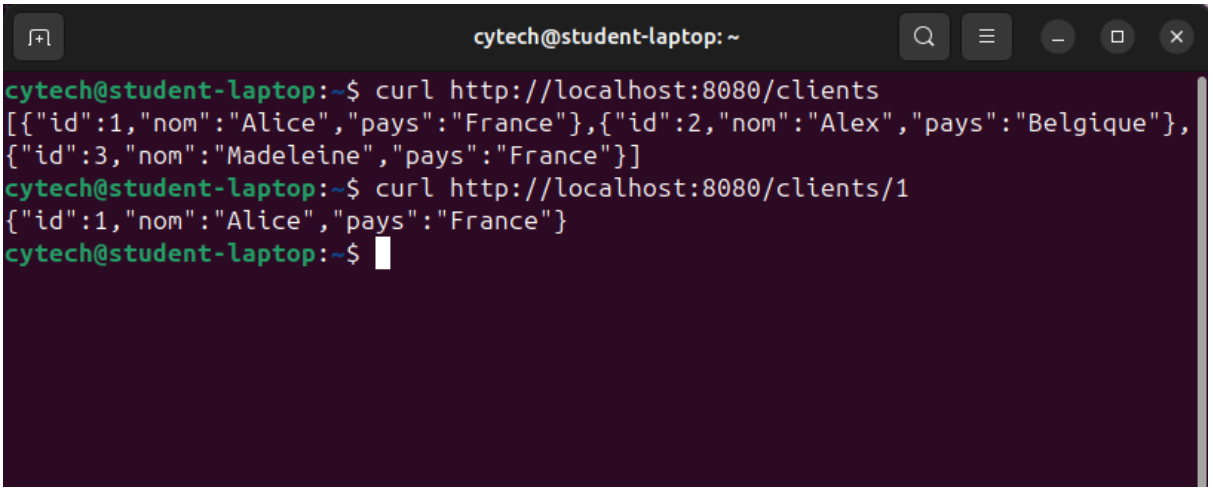

if __name__ == "__main__":
    app.run(host="0.0.0.0", port =8080)
```

Pour tester en local à lancer dans le terminal (toujours dans le dossier tp-microservices
cd clients
source ~/venv-clients/bin/activate
python app.py

si c'est la première fois pour créer les environnements virtuels :
cd clients
python3 -m venv ~/venv-clients
source ~/venv-clients/bin/activate
pip install-r requirements.txt
python app.py



Ouvrir un deuxième terminal :
curl http://localhost:8080/clients
curl http://localhost:8080/clients/1



une fois cela testé dans le terminal du dossier tp-microservices: crtl C, puis écrire deactivate

1.4 api rest du micro service commandes

Le microservice Commandes fonctionne sur le même principe. Il écoute aussi sur le port 8080 et expose deux routes :

GET /commandes : retourne toutes les commandes
GET /commandes/<id> : retourne une commande selon son id

le code de app.py (pas définitif, car modifié en 1.5, mais au stade du tp)

```
from flask import Flask, jsonify

app = Flask(__name__)

commandes = [
    {"id": 1, "client_id": 1, "produit": "PC", "montant": 1200},
    {"id": 2, "client_id": 2, "produit": "Livre", "montant": 30},
    {"id": 3, "client_id": 1, "produit": "Casque", "montant": 80},
]

@app.route("/commandes")
def get_commandes():
    return jsonify(commandes)

@app.route("/commandes/<int:id>")
def get_commande(id):
    for commande in commandes:
        if commande["id"] == id:
            return jsonify(commande)
    return jsonify({"erreur": "Commande introuvable"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

A lancer dans le terminal (donc dans le terminal tp-microservices, ou on est dans le sous dossier clients)
cd ../commandes
source ~/venv-commandes/bin/activate
python app.py

si c'est la premiere fois : création des environnements virtuels :
cd ../commandes
python3 -m venv ~/venv-commandes
source ~/venv-commandes/bin/activate
pip install-r requirements.txt
python app.py

```
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices/clients$ cd ../commandes
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices/commandes$ source ~/venv-commandes/bin/activate
(venv-commandes) cytech@student-laptop:/data/Documents/iCloudDrive/Documents/tra
vail/etudessup/6A/projet/tp-microservices/commandes$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://192.168.100.18:8080
Press CTRL+C to quit
```

et dans le deuxième terminal
curl http://localhost:8080/commandes
curl http://localhost:8080/commandes/1

```
cytech@student-laptop:~$ curl http://localhost:8080/commandes
[{"client_id":1,"id":1,"montant":1200,"produit":"PC"},{"client_id":2,"id":2,"mon
tant":30,"produit":"Livre"},{"client_id":1,"id":3,"montant":80,"produit":"Casque
"}]
cytech@student-laptop:~$ curl http://localhost:8080/commandes/1
{"client_id":1,"id":1,"montant":1200,"produit":"PC"}
cytech@student-laptop:~$
```

puis dans le terminal du dossier tp-microservices : ctrl C et deactivate

1.5 communication entre les micro services

Nous avons modifié le microservices commande pour qu'il appelle le microservice clients. Les modifications se concentrent dans le code de app.py. Désormais lors d'une requête GET /commandes/1 :

1. Récupération de la commande numéro 1
2. récupération de son client_id
3. appel du microservice clients avec une requête http, grâce à la bibliothèque requests
4. récuperation des informations du clients
5. Retour de la commande avec les informations du client.

L'adresse pour joindre clients est http://clients:8080. Ou clients est le nom du conteneur (précisé dans la partie 2). Ce fonctionnement est notamment testé en 2.1.B, une fois les deux micro-services dockerisés

```python
from flask import Flask, jsonify
import requests

app = Flask(__name__)
```

```python
#les questions du 1.4:
#les questions du 1.5 : communication entre micro services sont mis en commentaire, avec #1.5.1, #1.5.2...
# notre but est que le micro service commande puisse communiquer avec le micro service client
# Soit depuis la commande 1 par exemple, récupérer les infos associé au client de cette commande

#des commentaires ont aussi été rajoutées au dessus de la gestion des erreurs demandées en 2.2

#clients = nom du conteneur clients sur le réseau Docker
CLIENTS_URL = "http://clients:8080"


commandes = [
   {"id":1, "client_id": 1, "produit": "PC", "montant": 1200},
   {"id":2, "client_id": 2, "produit": "Livre", "montant": 30},
   {"id":3, "client_id": 1, "produit": "Casque", "montant": 80}
]

#1.4 : GET /commandes : retourne toutes les commandes
@app.route("/commandes")
def deg_commandes():
   return jsonify(commandes)

#1.4 : GET /commandes/<id> : retourne une commande (ajout du 1.5 plus les infos du client)
@app.route("/commandes/<int:id>")
def deg_commande(id):

   # 1.5.1 récuperer commande
   commande = None

   for c in commandes :

       if c["id"]== id:
           commande = c

   #2.2 gestion des erreurs : commande inexistante
   if commande is None:
       return jsonify({"erreur": "Commande introuvable"}), 404
```

```
    #1.5.2 récuperer client id pour #1.5.3 appeler le microservice
clients avec et #1.5.4 récupérer les infos du client
    try:
        r =
requests.get(f"{CLIENTS_URL}/clients/{commande['client_id']}",
timeout=3)

    #2.2 gestion des erreurs : microservice clients indisponible
    except requests.exceptions.RequestException:
        return jsonify({"erreur": "Service clients indisponible"}), 503

    #2.2 gestion des erreurs : client inexistant
    if r.status_code == 404:
        return jsonify({"erreur": "Client introuvable"}), 404

    # 1.5.5 renvoyer commande avec les infos du client
    return jsonify({
        "id": commande["id"],
        "produit": commande["produit"],
        "montant": commande["montant"],
        "client": r.json(),
    })


if __name__ == "__main__":
    app.run(host="0.0.0.0", port =8080)
```

2. Réseau docker

Dockeriser un microservice consiste à le faire tourner dans un conteneur Docker.
Il y a trois étapes :

1. Ecrire le dockerfile (la méthode, la recette...)
2. construire l'image (docker build)
3. lancer le conteneur docker run

2.1.A dockeriser le micro service clients

étape 1 : le Dockerfile dans clients ;

```
# on part d'une image contenant Python
FROM python:3.12-slim
# on travaille dans le dossier /app du conteneur
WORKDIR /app
# installation des dépendances
```

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# copie du code
COPY app.py .
# le programme écoute sur le port 8080
EXPOSE 8080
# commande lancée au démarrage du conteneur
CMD ["python", "app.py"]
```

étape 2 et 3 : construire image et lancer conteneur :
a lancer dans le terminal du dossier tp-microservices/commandes
cd ..
docker build -t microservice-clients ./clients
docker run -d -p 8080:8080 --name clients microservice-clients
curl http://localhost:8080/clients

-t microservice-clients donne un nom à l'image
--name clients donne un nom au conteneur
-p 8080:8080 relie le port 8080 de l'ordinateur au port 8080 du conteneur, pour pouvoir y accéder depuis l'extérieur.
-d lance le conteneur en arrière plan

```
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices/clients$ cd ..
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ docker build -t microservice-clients ./clients
[+] Building 0.5s (10/10) FINISHED                              docker:default
 => [internal] load build definition from Dockerfile                      0.0s
 => => transferring dockerfile: 264B                                      0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim       0.4s
 => [internal] load .dockerignore                                         0.0s
 => => transferring context: 2B                                           0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:a6e34c598f2467ed  0.0s
 => [internal] load build context                                         0.0s
 => => transferring context: 63B                                          0.0s
 => CACHED [2/5] WORKDIR /app                                             0.0s
 => CACHED [3/5] COPY requirements.txt .                                  0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt       0.0s
 => CACHED [5/5] COPY app.py .                                            0.0s
 => exporting to image                                                    0.0s
 => => exporting layers                                                   0.0s
 => => writing image sha256:b2d27c17070ce219050c6eda7e1a1aa268c419a1bb8ba  0.0s
 => => naming to docker.io/library/microservice-clients                   0.0s
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ docker run -d -p 8080:8080 --name clients microservice
-clients
2c3b8585ea6862e1dc912f67687cfd5ea9b1adc421363dd55764a7707f634619
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ curl http://localhost:8080/clients
[{"id":1,"nom":"Alice","pays":"France"},{"id":2,"nom":"Alex","pays":"Belgique"},
{"id":3,"nom":"Madeleine","pays":"France"}]
```

2.1.B dockeriser le micro service commandes

Le Dockerfile de commandes est identique à celui de clients. Il copie ainsi le requirements.txt et le app.py de son propre dosser.

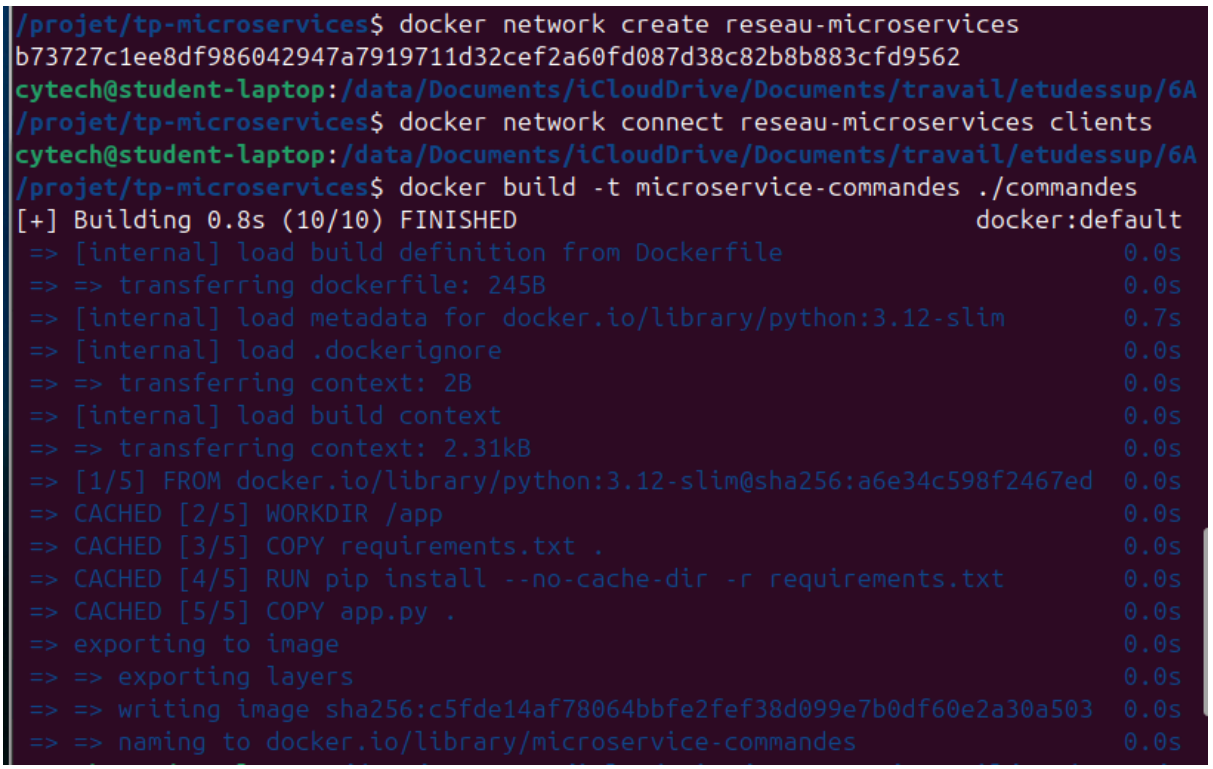
Pour que comandes puisse communiquer avec clients, des choix ont du être effectués :

- Nous avons créer un réseau Docker nommé reseau-microservices (nom qui représente le réseau qui relie nos microservices) et branché à deux conteneurs dessus. Sur un réseau créé par l'utilisateur, Docker permet aux conteneurs de se trouver par leur nom
- Nous avons défini l'adresse clients. Commandes joint clients avec http://clients:8080, soit le nom du conteneur et son port interne
- A l'intérieur les deux services écoutent sur 8080. Or sur ordinateur, un port ne peut être utilisé qu'une fois. Comme clients occupe déjà le 8080, commandes est accessible sur 8081.

Ainsi, l'utilisateur accède aux deux services depuis l'extérieur (localhost:8080 et localhost:8081), les conteneurs communiquent entre eux par le réseau "reseau-microservices"

A lancer dans le terminal dossier tp-microservices (seul docker build a besoin d'être à cet endroit mais autant tout lancer au même endroit, la deuxième ligne branche sur le réseau le conteneur clients, qui tourne déjà depuis le 2.1.A) :
docker network create reseau-microservices
docker network connect reseau-microservices clients
docker build -t microservice-commandes ./commandes
docker run -d -p 8081:8080 --name commandes --network reseau-microservices microservice-commandes

docker ps
curl http://localhost:8081/commandes/1



```
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ docker run -d -p 8081:8080 --name commandes --network
reseau-microservices microservice-commandes
e11ea3945b62c7cb91f1aa283032bcc577596226bfb110b7dfa35ccb20f4136f
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ docker ps
CONTAINER ID   IMAGE                  COMMAND           CREATED          STATU
S          PORTS                                     NAMES
e11ea3945b62   microservice-commandes   "python app.py"   11 seconds ago   Up 10
 seconds   0.0.0.0:8081->8080/tcp, :::8081->8080/tcp   commandes
2c3b8585ea68   microservice-clients     "python app.py"   16 minutes ago   Up 16
 minutes   0.0.0.0:8080->8080/tcp, :::8080->8080/tcp   clients
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$ curl http://localhost:8081/commandes/1
{"client":{"id":1,"nom":"Alice","pays":"France"},"id":1,"montant":1200,"produit"
:"PC"}
cytech@student-laptop:/data/Documents/iCloudDrive/Documents/travail/etudessup/6A
/projet/tp-microservices$
```

On voit les deux conteneurs en fonctionnement et la commande avec Alice, preuve de l'appel REST de commandes vers clients.

2.2 Gestion des erreurs

Il est demandé de gérer trois cas d'erreur :

- une commande inexistante (renvoie code http 404)
- une client inexistant (renvoie code http 404)
- si microservice clients pas disponible, commandes renvoit code 503 (service indisponible) au lieu de planter. Pour cela, l'appel à client, est placé dans un try , except.

```
cytech@student-laptop:~$ curl http://localhost:8080/clients/100
{"erreur":"Client introuvable"}
cytech@student-laptop:~$ curl http://localhost:8081/commandes/100
{"erreur":"Commande introuvable"}
cytech@student-laptop:~$ docker stop clients
clients
cytech@student-laptop:~$ curl http://localhost:8081/commandes/1
{"erreur":"Service clients indisponible"}
cytech@student-laptop:~$ docker start clients
clients
```

REMISE A ZERO : docker rm -f clients commandes
docker network rm reseau-microservices